

Etude de cas Optimisation 2

Adam DJAROUD ENSIIE

Décembre 2025

1 Le code

Le code est divisé en trois parties : une phase de prétraitement, l'implémentation du pl puis la résolution.

2 Analyse Structurelle et Pré-traitement

Avant d'initier la résolution, une phase de pré-traitement est effectuée via la fonction `analyser_structure`. Lors de cette phase, l'identification des ponts est réalisée : les arêtes dont la suppression déconnecte le graphe sont identifiées. Ces arêtes, appartenant nécessairement à tout arbre couvrant, sont injectées comme contraintes forcées dès le départ. Ensuite, un partitionnement des sommets est effectué : les sommets sont segmentés en deux ensembles. Le premier ensemble, noté V_{01} , regroupe les sommets de degré initial $d_G(v) \leq 2$, qui ne peuvent jamais devenir des sommets de branchement. Le second ensemble, noté V_2 , inclut les sommets de degré $d_G(v) > 2$, qui sont des candidats potentiels au branchement.

3 Le PL

À chaque itération, nous résolvons un programme linéaire en nombres entiers sur un graphe qui a été modifié par la boucle décrite ci-dessus et le ce même PL.

4 La Boucle Itérative

La contrainte de connexité globale est traitée de manière dynamique par une boucle `while` qui analyse la solution T fournie par le solveur.

4.1 Choix du Quota

Le paramètre `n_quota` détermine le nombre d'arêtes forcées pour relier les composantes isolées. Il est initialisé selon la racine carrée de la taille du graphe

:

$$n_quota = \max \left(1, \left\lfloor \frac{\sqrt{|V|}}{2} \right\rfloor \right) \quad (1)$$

Il est choisie ainsi pour être proportionnel à la taille du graphe et rendre l'algorithme plus efficace.

5 Comparaison des Approches de Reconnexion

L'évolution majeure entre la version initiale (*ilpsolver*) et la version optimisée (*ilpupdated*) réside dans l'algorithme de sélection des arêtes lors de la levée des contraintes de connexité.

5.1 Approche Initiale : Échantillonnage Aléatoire

Dans la première version, lorsqu'une solution T est déconnectée, l'algorithme identifie toutes les arêtes candidates reliant deux composantes distinctes. Un échantillon de taille n_quota est alors prélevé via `random.sample(candidates, sample_size)`. Cette méthode, bien que simple, ne garantit pas que les arêtes ajoutées ne vont pas dégrader l'objectif de l'arbre (en créant des branchements inutiles).

5.2 Approche Optimisée : Quota Intelligent

La version *Updated* introduit une fonction de scoring basée sur la topologie locale pour guider la reconnexion :

$$Score(e_{u,v}) = d_T(u) + d_T(v) \quad (2)$$

où $d_T(u)$ est le degré actuel du sommet u dans l'arbre partiel. Les arêtes candidates sont triées par score croissant. L'algorithme privilégie ainsi la connexion entre :

1. Deux sommets isolés (score 0).
2. Un sommet isolé et une feuille (score 1).
3. Deux feuilles (score 2).

Cette stratégie évite de forcer l'utilisation de sommets ayant déjà un degré ≥ 2 , retardant ainsi l'apparition de sommets de branchement.

6 Analyse des Résultats Expérimentaux

L'évaluation a été menée sur des instances de taille 20 à 100, 200 et 500. L'analyse des données de benchmark permet de quantifier l'apport de ce "quota intelligent".

6.1 Efficacité Algorithmique

- Moyenne Temps (Solver) : 7,8206 s
- Moyenne Temps (Updated) : 7,2687 s
- Gain de Temps Moyen : 10,41 %

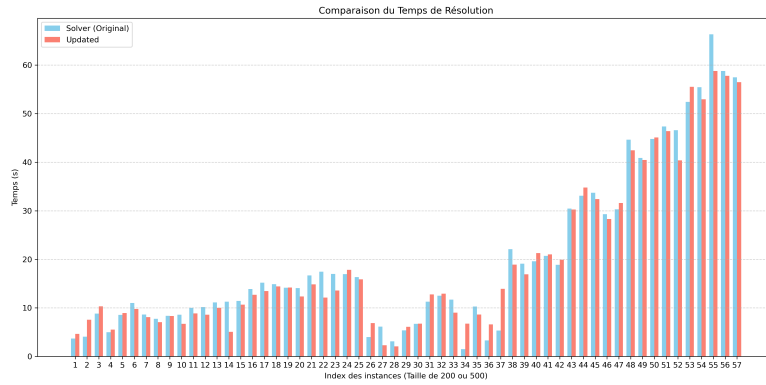


Figure 1: Évolution du temps de résolution par instance ($N = 200$).

L'amélioration du temps de calcul confirme que guider le solveur vers des connexions topologiquement logiques réduit le nombre de conflits internes au PLNE, facilitant une convergence plus rapide vers la faisabilité.

6.2 Optimisation de l'Objectif

- Branches moyennes (Solver) : 39,9834
- Branches moyennes (Updated) : 36,9725

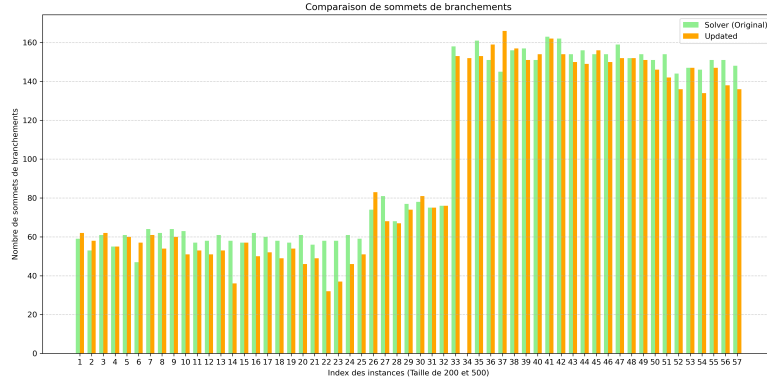


Figure 2: Comparaison du nombre de sommets de branchements.

Le passage d'une sélection aléatoire à une sélection par score minimal permet de réduire le nombre de branchements d'environ 7,5 %.

7 Conclusion

L'intégration d'une intelligence topologique dans le processus itératif de reconnexion s'est avérée être une stratégie robuste pour le problème MBVST. En remplaçant l'échantillonnage aléatoire par une fonction de score basée sur les degrés partiels, nous avons non seulement réduit le temps de calcul de 10,41 %, mais également amélioré la qualité de la solution optimale en minimisant les branchements forcés par les itérations.